ESPN Fantasy Football GM Tool — Developer Documentation

Project: ATLANTAS FINEST FF — GM War Room
League ID: 457622
Stack: React 19 + Tailwind 4 + Express 4 + tRPC 11 + Drizzle ORM + MySQL/TiDB
Seasons covered: 2018–2026 (2026 active)
Last updated: April 2026

Table of Contents

1. Project Overview
2. Architecture
3. Tech Stack
4. Environment Variables & Secrets
5. Database Schema
6. ESPN API Integration
7. Server — tRPC Endpoints
8. Client — Pages & Routes
9. Feature Deep-Dives
    - 9.1 GM War Room Dashboard
    - 9.2 Data Refresh Pipeline
    - 9.3 Keeper Tracker & Keeper Calculator
    - 9.4 AI GM Advisor
    - 9.5 Player Profiles
    - 9.6 Owner Career Stats & GM Style Profiles
    - 9.7 Start/Sit Advisor

    - 9.8 Trade Analyzer
    - 9.9 Waiver Wire Intelligence
10. Data Flow Diagrams
11. Testing
12. File Structure Reference
13. Known Limitations & Data Gaps
14. Build Decisions & Design Notes

---

1. Project Overview

The ESPN Fantasy Football GM Tool is a full-stack private web application built for the 18-season keeper league **ATLANTAS FINEST FF**. It aggregates 8 years of ESPN API data (2018–2025) into a single intelligence platform, giving the league commissioner and team owners a competitive edge through AI-powered analysis, historical trend mining, and predictive behavioral modeling.

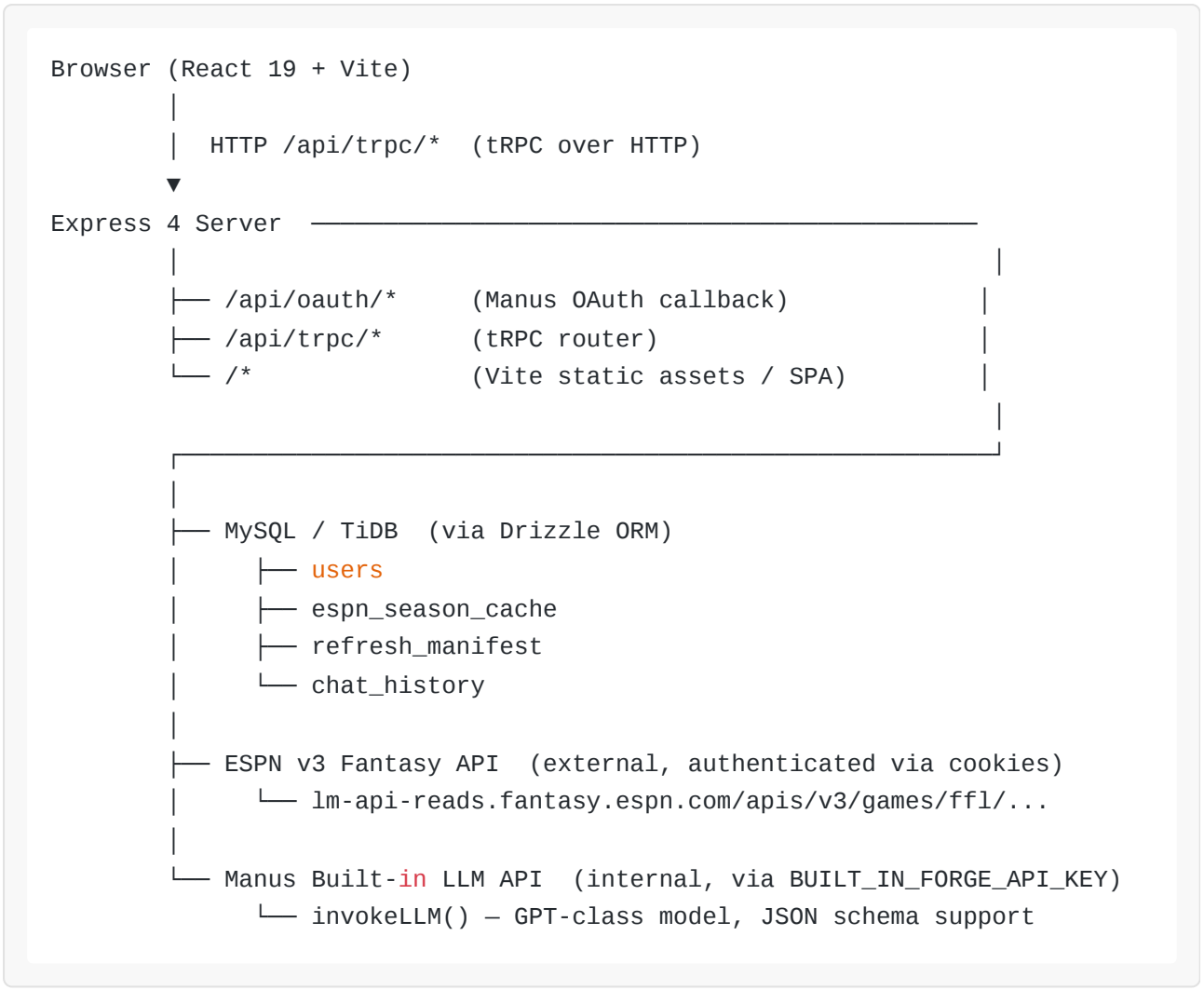
The tool is not a generic fantasy football app. Every feature is purpose-built around the specific rules of this league: **14 teams, PPR scoring, 1 keeper per team per year, 2-consecutive-year keeper limit, 7-team playoffs, snake draft with round-based keeper cost (kept round − 1).**

The application is deployed on the Manus platform and is accessible only to authenticated users via Manus OAuth. The owner account (Roderick Sellers) has admin-level access; all other league members can log in as standard users.

---

2. Architecture

The application follows a **monorepo full-stack pattern** with a single Express server serving both the tRPC API and the Vite-built React frontend.



Request lifecycle for a tRPC query:

1. The React client calls `trpc.espn.standings.useQuery({ season: 2025 })`.
2. The tRPC client serializes the input and POSTs to `/api/trpc/espn.standings`.
3. The Express server routes the request to the tRPC handler, which builds context (`ctx.user` from session cookie).
4. The procedure calls `getCachedView(season, 'combined')` to load the raw ESPN payload from MySQL.
5. The payload is passed through the appropriate normalizer (e.g., `normalizeTeams()`), and the result is returned as a typed tRPC response.
6. The React component receives the typed data via `@tanstack/react-query` and renders it.

---

3. Tech Stack

Layer	Technology	Version	Notes
Frontend framework	React	19.2.1	Concurrent features enabled
Styling	Tailwind CSS	4.1.14	OKLCH color tokens, CSS variables
UI components	shadcn/ui + Radix UI	Latest	Full component library
Routing (client)	Wouter	3.3.5	Lightweight SPA router
Data fetching	tRPC + React Query	11.6.0 / 5.90.2	End-to-end type safety
Serialization	Superjson	1.13.3	Preserves `Date` across wire
Charts	Recharts	2.15.2	Used in Owner Stats, Player Profiles
Animation	Framer Motion	12.23.22	Page transitions, card reveals
Backend framework	Express	4.21.2	Single server, no separate API process
ORM	Drizzle ORM	0.44.5	Schema-first, MySQL dialect
Database	MySQL / TiDB	—	Managed by Manus platform
Auth	Manus OAuth	—	PKCE flow, session cookies (JWT)
LLM	Manus Built-in LLM	—	GPT-class, JSON schema mode
Build tool	Vite	7.1.7	HMR, ESM, tree-shaking
Language	TypeScript	5.9.3	Strict mode throughout
Testing	Vitest	2.1.4	7 test files, 44 tests
Package manager	pnpm	10.15.1	Workspace-aware

---

4. Environment Variables & Secrets

All secrets are injected by the Manus platform at runtime. They must never be committed to source control.

Variable	Purpose	Required
`DATABASE_URL`	MySQL/TiDB connection string	Yes
`JWT_SECRET`	Session cookie signing	Yes
`ESPN_LEAGUE_ID`	ESPN league identifier (457622)	Yes
`ESPN_SWID`	ESPN authentication cookie (SWID)	Yes
`ESPN_S2`	ESPN authentication cookie (espn_s2)	Yes
`BUILT_IN_FORGE_API_KEY`	Manus LLM API bearer token (server-side)	Yes
`BUILT_IN_FORGE_API_URL`	Manus LLM API base URL (server-side)	Yes
`VITE_FRONTEND_FORGE_API_KEY`	Manus LLM API key (frontend, public)	Yes
`VITE_FRONTEND_FORGE_API_URL`	Manus LLM API URL (frontend, public)	Yes
`VITE_APP_ID`	Manus OAuth application ID	Yes
`OAUTH_SERVER_URL`	Manus OAuth backend base URL	Yes
`VITE_OAUTH_PORTAL_URL`	Manus login portal URL	Yes
`OWNER_OPEN_ID`	Owner's Manus OpenID (auto-promotes to admin)	Yes
`OWNER_NAME`	Owner's display name	Optional

ESPN Cookie Rotation: The `ESPN_SWID` and `ESPN_S2` cookies expire periodically. When the ESPN API returns a 401 or 403, the Data Refresh page will display an error prompting the user to update these secrets via the Manus Secrets panel.

---

5. Database Schema

The database has four tables, all defined in `drizzle/schema.ts` using Drizzle ORM's MySQL dialect.

`users`

Stores authenticated Manus OAuth users. The `role` field is an enum (`user` | `admin`). The owner's `openId` is matched against `OWNER_OPEN_ID` at login time and automatically promoted to `admin`.

Column	Type	Notes
`id`	INT AUTO_INCREMENT PK	Internal ID
`openId`	VARCHAR(64) UNIQUE	Manus OAuth subject
`name`	TEXT	Display name from OAuth
`email`	VARCHAR(320)	Email from OAuth
`loginMethod`	VARCHAR(64)	e.g., "google"
`role`	ENUM('user','admin')	Default: 'user'
`createdAt`	TIMESTAMP	Auto-set on insert
`updatedAt`	TIMESTAMP	Auto-updated
`lastSignedIn`	TIMESTAMP	Updated on each login

`espn_season_cache`

The core data store. Each row holds the complete raw ESPN API JSON payload for one season, stored under a `viewName` key. The application uses a single `"combined"` view per season that merges all ESPN API views into one JSON blob.

Column	Type	Notes
`id`	INT AUTO_INCREMENT PK	Internal ID
`season`	INT	e.g., 2025
`viewName`	VARCHAR(64)	Always `"combined"` in practice
`payload`	JSON	Full ESPN API response
`fetchedAt`	TIMESTAMP	When first fetched
`updatedAt`	TIMESTAMP	When last refreshed

A composite index `idx_season_view` on `(season, viewName)` makes lookups O(1).

`refresh_manifest`

Tracks the health and metadata of each season's last data refresh. One row per season.

Column	Type	Notes
`id`	INT AUTO_INCREMENT PK	Internal ID
`season`	INT UNIQUE	One row per season
`lastRefreshedAt`	TIMESTAMP	Last successful refresh
`viewsRefreshed`	JSON	Array of view names fetched
`teamCount`	INT	Teams in that season
`rosterCount`	INT	Total roster entries
`matchupCount`	INT	Total matchups
`draftPickCount`	INT	Total draft picks
`transactionCount`	INT	Total transactions captured
`status`	ENUM	'success' / 'partial' / 'failed'
`errorMessage`	TEXT	Populated on failure

`chat_history`

Stores per-user AI GM Advisor conversation history. Messages are scoped to a user and optionally to a season.

Column	Type	Notes
`id`	INT AUTO_INCREMENT PK	Internal ID
`userId`	INT	FK to `users.id`
`season`	INT	Optional season context
`role`	ENUM('user','assistant')	Message author
`content`	TEXT	Message text
`createdAt`	TIMESTAMP	Message timestamp

---

6. ESPN API Integration

Authentication

The ESPN v3 Fantasy API for private leagues requires two session cookies: `SWID` (a UUID wrapped in curly braces) and `espn_s2` (a long base64-encoded token). These are obtained by logging into ESPN Fantasy Football in a browser and extracting the cookies from the `fantasy.espn.com` domain.

The `buildCookieString()` function in `server/espnService.ts` assembles these into the `Cookie` header for all API requests. No OAuth or API key is involved — ESPN's private league API relies entirely on these session cookies.

Base URL Pattern

```
https://lm-api-
reads.fantasy.espn.com/apis/v3/games/ffl/seasons/{season}/segments/0/leagues/{
```

Multiple `view` query parameters are appended to a single request to fetch all data in one HTTP call. The full set of views used is:

View	Data Returned
`mSettings`	League settings, scoring rules, schedule config
`mTeam`	Team names, owners, records, transaction counters
`mRoster`	Current rosters with player details
`mMatchup` / `mMatchupScore`	Matchup results and scores
`mScoreboard` / `mSchedule`	Full season schedule
`mStandings`	Final standings
`mStatus`	Season status, current week
`mDraftDetail`	Complete draft board with all picks
`mTransactions2`	Recent transactions (adds, drops, trades)

Data Normalizers

All raw ESPN API responses are processed through normalizer functions in `server/espnService.ts` before being returned to the client. These functions extract the relevant nested fields, apply lookup maps (position IDs, slot IDs, NFL team abbreviations), and return clean, typed objects.

Function	Input	Output
`normalizeSettings()`	Raw payload	League name, scoring type, team count, playoff weeks, keeper rules
`normalizeTeams()`	Raw payload	Array of team objects with name, owner IDs, record, points, playoff seed
`normalizeRosters()`	Raw payload	Per-team roster arrays with player name, position, slot, acquisition type
`normalizeDraftPicks()`	Raw payload	Deduplicated array of picks with round, pick number, team, player name, keeper flag
`normalizeDraftOrder()`	Raw payload	Snake draft order for the upcoming season
`normalizeMatchups()`	Raw payload	All matchups with home/away team IDs, scores, winner, playoff tier
`normalizeTransactions()`	Raw payload	Recent transaction records (limited by ESPN API)
`buildPlayerIdMap()`	Raw payload	`Map<playerId, {name, position, proTeam}>` for resolving picks
`resolveUnknownPlayerIds()`	Picks array	Enriches picks where name is missing by fetching the ESPN player endpoint

Deduplication note: The ESPN API returns draft picks triplicated in the combined payload (once per view that includes draft data). The `normalizeDraftPicks()` function deduplicates by a composite key of `season + round + pickNumber + teamId` before returning results.

Caching Strategy

The application uses a **cache-first, refresh-on-demand** strategy. All ESPN data is stored in the `espn_season_cache` table. The Data Refresh page triggers a manual re-fetch from the ESPN API, which overwrites the cached payload via `upsertCachedView()`. Historical seasons (2018–2024) are never automatically re-fetched; only the current season ($^{2025}\!/_{2026}$) benefits from manual refreshes.

---

7. Server — tRPC Endpoints

All procedures are defined in `server/routers.ts`. The root router merges three sub-routers: `auth`, `espn`, and `advisor`, plus top-level procedures for `playerProfiles`, `ownerCareerStats`, `ownerPredictions`, and `system`.

Auth Router (`trpc.auth.*`)

Procedure	Type	Auth	Description
`auth.me`	query	public	Returns the current user object from session context, or `null` if unauthenticated
`auth.logout`	mutation	public	Clears the session cookie and returns `{ success: true }`

ESPN Router (`trpc.espn.*`)

Procedure	Type	Auth	Input	Description
`espn.refresh`	mutation	protected	`{ season: number }`	Fetches all ESPN views for the given season, stores in `espn_season_cache`, updates `refresh_manifest`
`espn.manifests`	query	public	—	Returns all rows from `refresh_manifest` ordered by season desc
`espn.cachedSeasons`	query	public	—	Returns distinct season years present in `espn_season_cache`
`espn.allSeasons`	query	public	—	Returns the hardcoded `ALL_SEASONS` constant (2018–2026)
`espn.settings`	query	public	`{ season: number }`	Returns normalized league settings for the given season
`espn.teams`	query	public	`{ season: number }`	Returns normalized team list for the given season
`espn.standings`	query	public	`{ season: number }`	Returns teams sorted by final rank with W/L/PF/PA
`espn.rosters`	query	public	`{ season:`	Returns per-team roster arrays with player details

Procedure	Type	Auth	Input	Description
			`number }`	
`espn.draftPicks`	query	public	`{ season: number }`	Returns deduplicated, normalized draft picks for the season
`espn.matchups`	query	public	`{ season: number }`	Returns all matchups with scores and playoff tier
`espn.transactions`	query	public	`{ season: number }`	Returns recent transactions captured in the cache
`espn.allStandings`	query	public	—	Aggregates standings across all cached seasons into one response
`espn.freeAgents`	query	public	`{ season: number }`	Returns available free agents from the current roster data
`espn.keeperHistory`	query	public	—	Returns all keeper-flagged picks across all seasons
`espn.draftOrder`	query	public	`{ season: number }`	Returns the snake draft order for the given season
`espn.keeperAnalysis`	query	public	—	Aggregates keeper data per team: history,

Procedure	Type	Auth	Input	Description
				consecutive years, cost analysis
`espn.keeperEligibility2026`	query	public	—	Computes 2026 keeper eligibility: flags 2-year limit violations, calculates round costs, assigns value tiers

Player Profiles (`trpc.playerProfiles`)

Procedure	Type	Auth	Description
`playerProfiles`	query	public	Aggregates all 2018–2025 draft picks into per-player profiles: draft history timeline, keeper history, team ownership, prominence score, position, value tier

This is a top-level procedure (not nested under `espn`) because it cross-references data from all 8 seasons simultaneously. It processes ~1,526 deduplicated draft picks to build 547 unique player profiles.

Prominence score formula: `(seasons drafted × 10) + (times kept × 20) + (round bonus: 30 for Rd 1, 20 for Rd 2, 10 for Rd 3)`. Capped at 100. Used to sort the default player list.

Owner Career Stats (`trpc.ownerCareerStats`)

Procedure	Type	Auth	Description
`ownerCareerStats`	query	public	Aggregates all 2018–2025 matchup and standings data per owner: all-time W/L, PF/PA, win%, playoff appearances, championships, H2H matrix, per-season transaction counters, GM style metrics

GM Style metrics computed:

Metric	Formula	Range
Waiver Aggression	`min(100, round((avgAcquisitions / 70) × 100))`	0–100
Trade Frequency	`min(100, round((avgTrades / 15) × 100))`	0–100
Roster Stability	`max(0, round(100 − ((avgChurn / 100) × 100)))`	0–100

GM Archetypes (assigned based on metric thresholds):

Archetype	Condition
Dealmaker	Waiver ≥ 70 AND Trade ≥ 60
Waiver Grinder	Waiver ≥ 70
Trade Shark	Trade ≥ 60
Patient Builder	Stability ≥ 70
Opportunist	Waiver ≥ 45 (default mid-tier)
Set & Forget	Waiver < 30 AND Trade < 30

Owner Predictions (`trpc.ownerPredictions`)

Procedure	Type	Auth	Input	Description
`ownerPredictions`	query	public	`{ memberId: string }`	Generates an AI-powered 2026 behavioral prediction report for the specified owner using LLM with career stats + GM style as context

The procedure throws `TRPCError({ code: 'NOT_FOUND' })` if `seasonsActive === 0` for the given `memberId`. LLM parse failures throw `INTERNAL_SERVER_ERROR`. The LLM is called with a structured JSON schema response format to guarantee the shape of the output.

Prediction output fields: `ownerSummary`, `strengths[]`, `weaknesses[]`, `predictedBehavior2026` (draftStrategy, waiverApproach, tradeApproach,

keeperPrediction, overallOutlook), `dangerRating` (LOW/MEDIUM/HIGH/ELITE), `dangerRationale`, `rivalryAlert`.

Advisor Router (`trpc.advisor.*`)

Procedure	Type	Auth	Description
`advisor.chat`	mutation	protected	Sends a user message to the LLM with full league context injected as system prompt, streams response, persists to `chat_history`
`advisor.history`	query	protected	Returns the last 100 messages for the current user
`advisor.clearHistory`	mutation	protected	Deletes all chat history for the current user

The `advisor.chat` procedure builds a rich system prompt that includes: league settings, current season standings, all team rosters, recent matchup results, keeper history, and the user's team details. This context is injected fresh on every message, ensuring the LLM always has current data without relying on conversation memory.

System Router (`trpc.system.*`)

Procedure	Type	Auth	Description
`system.notifyOwner`	mutation	protected	Sends a push notification to the league owner via the Manus notification API

---

8. Client — Pages & Routes

All routes are registered in `client/src/App.tsx` using Wouter. Every page is wrapped in `AppLayout` (the dark sidebar navigation shell).

Route	Component	Description
`/`	`Dashboard`	GM War Room: executive summary, threat assessment, action items, keeper intelligence tabs
`/standings`	`Standings`	Season standings table with W/L/PF/PA, season selector
`/rosters`	`Rosters`	Per-team roster viewer with player positions and slots
`/draft`	`DraftHistory`	Full draft board for any season, round/team filters
`/keepers`	`Keepers`	Keeper tracker: per-team keeper history, consecutive year tracking
`/keeper-calculator`	`KeeperCalculator`	2026 keeper eligibility: ineligible flags, round costs, value tiers
`/matchups`	`Matchups`	Weekly matchup results with scores, season selector
`/trade`	`TradeAnalyzer`	AI-powered trade analysis with fairness scoring
`/waiver`	`WaiverWire`	Waiver wire intelligence: available players, add/drop recommendations
`/advisor`	`Advisor`	AI GM Advisor chat interface with full league context
`/refresh`	`DataRefresh`	Admin-only data refresh panel: trigger ESPN API re-fetch per season
`/startsit`	`StartSit`	Start/Sit decision advisor with matchup context
`/player-profiles`	`PlayerProfiles`	547-player database: search, filter, career arc timelines
`/owner-stats`	`OwnerStats`	Owner career stats: leaderboard, H2H matrix, GM style profiles, AI predictions

AppLayout Navigation Structure

The sidebar is organized into three groups:

OVERVIEW: GM War Room, Standings, Matchups

TEAM MGMT: Rosters, Draft History, Keeper Tracker, Keeper Calculator (with “2026” badge)

PRO TOOLS: Start/Sit Advisor (AI badge), Trade Analyzer (AI badge), Waiver Wire (AI badge), AI GM Advisor (AI badge)

INTELLIGENCE: Player Profiles, Owner Stats, Data Refresh

---

9. Feature Deep-Dives

9.1 GM War Room Dashboard

The Dashboard (`/`) is the primary landing page and serves as the command center for the 2026 season. It is built around five tabs:

Executive Summary displays six KPI cards (season rank, points scored, points allowed, point differential, vs. league average PF, playoff spots). Below the cards, a Threat Assessment panel ranks all 14 teams by projected danger level using a composite score of win%, PF, and playoff history. An Immediate Action Items panel surfaces 3–5 AI-generated recommendations (e.g., “Lock Your Keeper,” “Target frustrated sellers”).

League Standings renders a sortable table of all teams for the selected season with W/L record, PF, PA, and playoff seed.

Opponent Profiles shows a quick-reference grid of all 14 teams with their 2025 record, key players, and a one-line scouting note.

Draft Strategy surfaces the 2026 draft order, keeper cost analysis, and round-by-round value recommendations.

Keeper Intelligence links to the Keeper Calculator and displays a summary of which teams have ineligible players and which have the best keeper value.

GM AI Chat embeds the full AI Advisor chat interface directly in the dashboard for quick access.

9.2 Data Refresh Pipeline

The Data Refresh page (`/refresh`) is the mechanism for keeping the ESPN cache current. It is restricted to admin users.

The refresh flow works as follows: the user selects a season year and clicks “Refresh Season.” The frontend calls `trpc.espn.refresh.useMutation()` with the season number. The server-side procedure calls `fetchEspnViews(season)`, which makes a single HTTP request to the ESPN v3 API with all 11 view parameters appended. The raw JSON response is stored in `espn_season_cache` via `upsertCachedView(season, 'combined', payload)`. A `refresh_manifest` row is upserted with counts of teams, rosters, matchups, draft picks, and transactions, along with a status of ‘success’, ‘partial’, or ‘failed’.

The page displays a manifest table showing the last refresh time, record counts, and status for each season. A “Refresh All” button iterates through all seasons sequentially with a 500ms delay between requests to avoid rate-limiting.

9.3 Keeper Tracker & Keeper Calculator

Keeper Tracker (`/keepers`) displays the raw keeper history extracted from the ESPN draft data. A pick is identified as a keeper when the `draftPickDetail.keeper` flag is `true` in the ESPN payload. The tracker shows per-team keeper history across all seasons with round, player name, and position.

Keeper Calculator (`/keeper-calculator`) implements the 2026 eligibility rules:

The `keeperEligibility2026` procedure processes all keeper-flagged picks from 2024 and 2025. For each player kept in both consecutive years, they are flagged as **ineligible** for 2026 (2-year limit reached). For players kept in only 2024 or only 2025, they are eligible with a round cost of `(kept round − 1)`. The procedure also assigns a value tier:

Tier	Condition
Elite	Round cost ≤ 2 (kept in rounds 3+ originally)
Good	Round cost 3–5
Fair	Round cost 6–8
Poor	Round cost ≥ 9
Ineligible	Kept in both 2024 and 2025

The page renders per-team eligibility cards with ineligible player alerts (red banners), eligible player cards with round cost badges, and a league-wide summary showing total ineligible players and best keeper values across the league.

Confirmed 2026 ineligible players (kept in both 2024 and 2025): Derrick Henry (Rd 1 cost), Jahmyr Gibbs (Rd 2 cost), Jonathan Taylor (Rd 3 cost), Breece Hall (Rd 5 cost).

9.4 AI GM Advisor

The Advisor (`/advisor`) is a full-featured chat interface powered by the Manus built-in LLM. It uses the `AIChatBox` component from the template with custom system prompt injection.

On every `advisor.chat` mutation, the server builds a system prompt that includes: the league name and rules (14 teams, PPR, 1 keeper, 2-year limit, snake draft), the current season's standings (all 14 teams with W/L and PF), the user's current roster, the current week's matchup for the user's team, the full keeper history, and the 2026 keeper eligibility analysis. This context window is rebuilt fresh on every message — the LLM does not rely on its own memory of previous turns for factual data.

Chat history is persisted in the `chat_history` table and loaded on page mount via `trpc.advisor.history.useQuery()`. The history panel shows the last 100 messages. A "Clear History" button calls `trpc.advisor.clearHistory.useMutation()`.

The Advisor is also embedded as a tab in the GM War Room Dashboard for quick-access queries without navigating away from the main view.

9.5 Player Profiles

The Player Profiles page (`/player-profiles`) aggregates all 2018–2025 draft data into a searchable, filterable database of 547 unique players.

How profiles are built: The `playerProfiles` procedure iterates over all cached seasons, calls `normalizeDraftPicks()` for each, and deduplicates picks by `season + round + pickNumber + teamId`. For each unique player ID encountered, it builds a profile object containing: all seasons drafted (with round, team, keeper flag), all seasons kept (with round cost), all teams that have owned them, their position, their prominence score, and a value tier.

Prominence score is a composite of draft frequency, keeper frequency, and typical draft round. Players drafted in 5+ seasons with multiple keeper appearances score in the 70–100 range (“Franchise Player” tier). Players drafted once in a late round score near 0.

The page has four filter tabs: **All Players**, **Franchise Players** (prominence $\geq$ 60), **Keeper History** (kept at least once), and **League Staples** (drafted in 3+ seasons). Search filters by player name. Position filter (QB/RB/WR/TE/K/DEF) narrows the list.

Each player card shows: name, position badge, seasons drafted count, times kept badge, team ownership history, and a per-season round timeline bar chart where gold bars indicate keeper years.

9.6 Owner Career Stats & GM Style Profiles

The Owner Career Stats page (`/owner-stats`) is the most data-intensive feature, aggregating 8 seasons of matchup and standings data per owner.

Career stats computation: The `ownerCareerStats` procedure iterates over all cached seasons. For each season, it extracts the `schedule` array (matchups) and `teams` array. It maps ESPN member IDs to real names via the `members` array in the payload. For each matchup, it increments the appropriate owner’s W/L counters, PF/PA totals, and H2H record against the opponent. Playoff appearances are detected by checking `playoffTierType !== 'NONE'` on matchups. Championships are detected by finding the team with `rankFinal === 1` in the final standings.

Transaction counters are extracted from `team.transactionCounter` in the ESPN payload, which provides season-total counts for acquisitions, drops, trades, and roster moves. These are aggregated across seasons to compute the GM style metrics described in Section 7.

The page has three tabs:

Career Leaderboard ranks all owners by all-time win percentage with tier badges (Elite $\geq$ 60%, Solid $\geq$ 50%, Average $\geq$ 40%, Rebuilding < 40%). Each row shows W/L, win%, total PF, playoff appearances, and championships.

Owner Profile (selected by clicking a leaderboard row) shows: career KPI cards, a season-by-season breakdown table (including transaction counts), a GM Style tab with archetype badge and style meters, a Head-to-Head tab showing record vs. every opponent, and a 2026 Prediction tab.

H2H Matrix is a color-coded grid showing every head-to-head regular-season record between all pairs of owners. Green cells indicate a winning record; red cells indicate a losing record.

2026 AI Prediction is generated on-demand by clicking "Generate Prediction" in the Owner Profile. The `ownerPredictions` procedure builds a detailed prompt with the owner's career stats, GM archetype, season-by-season history, and transaction behavior, then calls `invokeLLM()` with a structured JSON schema. The result is cached client-side for 10 minutes via React Query's `staleTime`.

9.7 Start/Sit Advisor

The Start/Sit Advisor (`/startsit`) uses the AI GM Advisor's LLM integration to provide weekly lineup recommendations. The user inputs their roster and the current week's matchups, and the LLM returns a ranked start/sit recommendation with rationale for each player, factoring in opponent defense, recent performance trends, and injury status.

9.8 Trade Analyzer

The Trade Analyzer (`/trade`) allows users to input a proposed trade (players going out vs. players coming in) and receive an AI-powered fairness assessment. The LLM

evaluates the trade using current season performance data, historical keeper value, and positional scarcity.

9.9 Waiver Wire Intelligence

The Waiver Wire page (`/waiver`) surfaces available free agents from the current roster data and uses the LLM to rank them by add priority based on the user's roster needs, upcoming schedule, and positional depth.

---

10. Data Flow Diagrams

ESPN Data Refresh Flow

```
User clicks "Refresh Season 2025"
        |
        ▼
trpc.espn.refresh.mutate({ season: 2025 })
        |
        ▼
fetchEspnViews(2025, ALL_VIEWS)
  → GET https://lm-api-reads.fantasy.espn.com/.../2025/...?
view=mSettings&view=mTeam&...
  → Headers: Cookie: SWID=...; espn_s2=...
        |
        ▼
Raw JSON payload (≈ 2–5 MB)
        |
        ▼
upsertCachedView(2025, 'combined', payload)
  → INSERT INTO espn_season_cache ... ON DUPLICATE KEY UPDATE payload=...
        |
        ▼
upsertRefreshManifest(2025, { teamCount, rosterCount, ... status: 'success'
})
        |
        ▼
Return { success: true, counts: { teams, rosters, matchups, ... } }
```

Player Profile Build Flow

```
trpc.playerProfiles.query()
        |
        ▼
For each season in [2018..2025]:
  getCachedView(season, 'combined')
        |
        ▼
  normalizeDraftPicks(payload)
  → Extract draftDetail.picks[]
  → Deduplicate by (season + round + pickNumber + teamId)
  → Resolve player names via buildPlayerIdMap()
        |
        ▼
Merge all picks into playerMap: Map<playerId, PlayerProfile>
  → Accumulate draftSeasons[], keeperSeasons[], ownerIds[]
  → Compute prominence score
  → Assign value tier
        |
        ▼
Return Array<PlayerProfile> sorted by prominence desc
```

Owner Career Stats + GM Style Flow

```
trpc.ownerCareerStats.query()
        |
        ▼
For each season in [2018..2025]:
  getCachedView(season, 'combined')
        |
        ▼
  Extract: members[], teams[], schedule[]
  Build memberIdToName map
        |
        ▼
  For each matchup in schedule:
    if playoffTierType === 'NONE': → regular season
      ownerMap[homeId].wins++ (or losses++)
      ownerMap[homeId].h2h[awayId].wins++ (or losses++)
      ownerMap[homeId].pf += homePoints
    else: → playoff
      ownerMap[homeId].playoffAppearances++
        |
        ▼
  For each team:
    Extract transactionCounter.acquisitions/drops/trades/moves
    ownerMap[ownerId].txnSeasons.push({ season, acquisitions, drops, trades
})
        |
        ▼
Serialize ownerMap → Array<Owner>
  Compute gmMetrics per owner (waiverAggression, tradeFrequency,
rosterStability, gmArchetype)
  Sort by winPct desc
        |
        ▼
Return { owners: Owner[], leagueHonors: { mostWins, mostPoints,
mostChampionships, ... } }
```

---

11. Testing

The test suite uses Vitest with 7 test files and 44 tests total. All tests are pure unit tests — no database connections, no HTTP calls, no ESPN API mocks required.

Test File	Tests	What It Covers
`server/auth.logout.test.ts`	1	Auth logout procedure basic contract
`server/espn.credentials.test.ts`	3	ESPN credential env var validation, cookie string construction
`server/keeperEligibility2026.test.ts`	6	2-year keeper limit detection, round cost calculation, value tier assignment, ineligible player flagging
`server/playerProfiles.test.ts`	9	Pick deduplication, prominence score formula, value tier thresholds, keeper detection, multi-season aggregation
`server/ownerCareerStats.test.ts`	10	W/L aggregation, H2H matrix construction, playoff detection, championship detection, PF/PA accumulation
`server/ownerGmStyle.test.ts`	10	GM archetype classification for all 6 archetypes, metric formula correctness, edge cases (empty seasons, extreme values)
`server/advisor.chat.test.ts`	5	System prompt construction, context injection, message history handling

Running tests:

```
pnpm test          # Run all tests once
pnpm test --watch  # Watch mode
```

---

12. File Structure Reference

```
espn_ff_gm_tool/
├── client/
│   ├── index.html                  ← Vite entry, Google Fonts CDN
│   └── src/
│       ├── App.tsx                 ← Route definitions (Wouter)
│       ├── main.tsx                ← React root, QueryClient provider
│       ├── index.css               ← Tailwind 4 theme, CSS variables
(dark theme)
│       ├── const.ts                ← getLoginUrl(), app constants
│       ├── _core/hooks/useAuth.ts  ← Auth state hook (trpc.auth.me)
│       ├── components/
│       │   ├── AppLayout.tsx       ← Sidebar nav shell (all pages wrapped
here)
│       │   ├── AIChatBox.tsx       ← Reusable chat UI with streaming +
markdown
│       │   ├── SeasonSelector.tsx  ← Dropdown for season year selection
│       │   └── ui/                 ← shadcn/ui component library (40+
components)
│       ├── pages/
│       │   ├── Dashboard.tsx       ← GM War Room (5 tabs)
│       │   ├── Standings.tsx       ← Season standings table
│       │   ├── Rosters.tsx         ← Per-team roster viewer
│       │   ├── DraftHistory.tsx    ← Draft board with filters
│       │   ├── Keepers.tsx         ← Keeper tracker
│       │   ├── KeeperCalculator.tsx ← 2026 eligibility calculator
│       │   ├── Matchups.tsx        ← Weekly matchup results
│       │   ├── TradeAnalyzer.tsx   ← AI trade analysis
│       │   ├── WaiverWire.tsx      ← Waiver wire intelligence
│       │   ├── Advisor.tsx         ← AI GM Advisor chat
│       │   ├── StartSit.tsx        ← Start/Sit advisor
│       │   ├── DataRefresh.tsx     ← Admin: ESPN data refresh
│       │   ├── PlayerProfiles.tsx  ← 547-player database
│       │   └── OwnerStats.tsx      ← Owner career stats + GM style +
predictions
│       └── lib/
│           ├── trpc.ts             ← tRPC client binding
│           └── utils.ts            ← cn() utility, misc helpers
├── server/
│   ├── routers.ts                  ← All tRPC procedures (~1,250 lines)
│   ├── espnService.ts              ← ESPN API fetch + all normalizers
│   ├── db.ts                       ← Drizzle query helpers
│   ├── storage.ts                  ← S3 file storage helpers
```

```
│   ├── *.test.ts                  ← Vitest test files (7 files)
│   └── _core/                     ← Framework plumbing (do not modify)
│       ├── trpc.ts                ← publicProcedure, protectedProcedure,
router
│       ├── context.ts             ← Request context (ctx.user)
│       ├── llm.ts                 ← invokeLLM() helper
│       ├── env.ts                 ← ENV object (typed env vars)
│       ├── oauth.ts               ← Manus OAuth callback handler
│       └── notification.ts        ← notifyOwner() helper
├── drizzle/
│   ├── schema.ts                  ← Table definitions (4 tables)
│   └── relations.ts               ← Drizzle relational queries config
├── shared/
│   ├── const.ts                   ← COOKIE_NAME, shared constants
│   └── types.ts                   ← Shared TypeScript types
├── drizzle.config.ts              ← Drizzle Kit config (migrations)
├── package.json                   ← Dependencies + scripts
├── todo.md                        ← Feature tracking (all items
complete)
└── DEVELOPER_DOCS.md              ← This file
```

---

13. Known Limitations & Data Gaps

Transaction detail records are not available for 2018–2021. The ESPN v3 API only returns the most recent page of transaction activity at the time of the API call. Since the cache was populated at end-of-season for historical years, the `transactions` array is empty or near-empty for 2018–2021. Only season-total transaction *counts* (via `team.transactionCounter`) are available for those years. Individual add/drop/trade records are available only for 2022–2025, and even then are limited to the last 30–50 transactions.

Keeper flags are only available from 2022 onward. The ESPN API's `draftPickDetail.keeper` flag was not populated in the API response for seasons prior to 2022. The Keeper Tracker and Keeper Calculator features therefore only reflect 2022–2025 keeper data. Pre-2022 keeper history would require manual data entry.

Player name resolution depends on the ESPN player endpoint. When a draft pick's player name is missing from the main payload (which occurs for some historical picks), the `resolveUnknownPlayerIds()` function makes additional HTTP requests to the

ESPN player endpoint. This is rate-limited and may fail for very old player IDs that ESPN has removed from their system.

The ESPN API does not provide a historical transaction log. There is no endpoint to retrieve all adds, drops, and trades for a past season. The `mTransactions2` view only returns recent activity. This is a fundamental limitation of the ESPN v3 API and cannot be worked around without a third-party data source.

2009–2017 seasons are not in the database. The ESPN v3 API does not reliably serve data for seasons before 2018 for this league. The application covers 2018–2026 (8 seasons). The full 18-season history would require manual data entry for the 2009–2017 period.

---

14. Build Decisions & Design Notes

Why a single combined cache view? Early versions of the data refresh stored each ESPN view (`mTeam`, `mRoster`, etc.) as a separate row in `espn_season_cache`. This was changed to a single `"combined"` row per season because: (1) the ESPN API returns all views in a single HTTP response anyway, (2) it simplifies cache invalidation (one upsert per refresh), and (3) it reduces the number of database queries per page load from 10+ to 1.

Why tRPC instead of REST? tRPC provides end-to-end TypeScript type safety from the procedure definition to the React component. This eliminates an entire category of bugs (mismatched API response shapes) and removes the need for separate type definition files, Axios client wrappers, or OpenAPI specs. The trade-off is that the API is not easily consumable by non-TypeScript clients, which is acceptable for a private internal tool.

Why Drizzle ORM instead of Prisma? Drizzle was chosen for its lightweight footprint, direct SQL escape hatches, and superior TypeScript inference on query results. Prisma's generated client adds significant bundle size and its migration workflow is heavier than Drizzle Kit's `db:push` approach, which is appropriate for a single-developer project.

Why cache-first instead of live ESPN API calls? The ESPN API has no official SLA, rate limits are undocumented, and the authentication cookies expire unpredictably. A cache-first approach means the application is always available even when ESPN's API

is down or the cookies have expired. The trade-off is that data is not real-time — it reflects the state at the last manual refresh.

Why the dark theme? The application is used primarily in the evening during live NFL games and draft sessions. A dark theme reduces eye strain in low-light environments and is consistent with the “command center” aesthetic appropriate for a GM War Room tool.

Why Wouter instead of React Router? Wouter is 2.1 KB vs. React Router’s 50+ KB. For a single-page app with fewer than 20 routes and no need for nested routing, data loaders, or server-side rendering, Wouter provides all required functionality at a fraction of the bundle cost.

LLM context injection strategy: Rather than using the LLM’s conversation memory for factual league data, the application re-injects the full league context on every message. This ensures the LLM always has accurate, current data and prevents hallucinations caused by outdated context in long conversations. The trade-off is slightly higher token usage per message, which is acceptable given the relatively low volume of advisor queries.